

NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL
Department of Computer Science and Engineering

Course Code	CS1303
Course Title	Full Stack Development
Assignment No.	3
Title	Backend Integration for the Portfolio Website (Node.js / Express)
Duration	2 Weeks
Date of Issue	21 Aug 2026
Date of Submission	3 Sept 2026 , 11:59 PM
Maximum Marks	
Weightage	

1. Problem Statement

This assignment extends the React Portfolio Website built in Assignment 2 by adding a real Node.js/Express backend. The static project data and client-side-only contact form from Assignment 2 will now be served and handled by a live API, giving you hands-on experience connecting a React frontend to a backend service — the natural next step flagged in the previous assignment.

You must build on top of your existing Assignment 2 codebase. This is not a new project: your Navbar, routing, theme toggle, and page structure must continue to work exactly as before; only the data source for projects and the destination of the contact form submit are changing.

2. Learning Objectives

- Build and run a minimal Express server with environment-based configuration.
- Design REST endpoints that an existing frontend can consume without redesigning its UI.
- Use fetch/axios inside useEffect to load remote data, and manage loading and error states correctly.
- Validate incoming data on the server and surface server-side errors meaningfully on the client.
- Enable CORS and understand why cross-origin requests fail without it.

3. Tasks — Backend (Express Server)

Each task below states exactly what must work and how it will be checked.

B1. Express Server Setup & Health Check

Create a Node.js/Express server in a /server folder (within the same repository as your Assignment 2 frontend), listening on a port defined by an environment variable. Expose a base route that confirms the API is running.

Evaluation criteria:

- GET / returns HTTP 200 with a JSON body such as { "status": "ok" }.
- Server port is read from a PORT variable in a .env file (not hardcoded).
- Server starts with a single documented command (e.g., npm start or npm run dev) from within /server.

B2. GET /api/projects — Serve Project List

Move the project data currently hardcoded in src/data/projects.js into the backend, and expose it through this endpoint. Each project object must retain the same fields your ProjectCard component already expects (id, title, description, techStack, image, link).

Evaluation criteria:

- Returns HTTP 200 with a JSON array of at least 3 project objects.
- Every object contains all fields required by the existing ProjectCard props — no field renamed or dropped.
- Data is stored server-side (in a JS/JSON file or in-memory array), not duplicated in the frontend anymore.

B3. GET /api/projects/:id — Serve a Single Project

Add a route that returns one project matching the given id, to support the existing dynamic /projects/:projectId detail page.

Evaluation criteria:

- Valid id returns HTTP 200 with the matching project object.
- Non-existent id returns HTTP 404 with a JSON body such as { "error": "Project not found" } — not an HTML error page.

B4. POST /api/contact — Handle Contact Form Submissions

Add an endpoint that accepts { name, email, message } from the existing Contact page, validates it server-side, and persists valid submissions (in-memory array, a JSON file, or a database of your choice).

Evaluation criteria:

- Missing name, email, or message returns HTTP 400 with a field-specific error message.
- An invalid email format (e.g., missing "@") returns HTTP 400 with a clear error message.
- A valid submission returns HTTP 201 with a confirmation payload, and is actually persisted (verifiable by re-reading storage or via task B5).

B5. GET /api/contact — List Submissions (for verification)

Add a simple endpoint that returns all stored contact submissions, so that stored data from B4 can be verified during evaluation. No authentication is required for this assignment, but this must be clearly noted in the README as an open endpoint.

Evaluation criteria:

- Returns HTTP 200 with a JSON array of all submissions received so far, including ones sent from the live frontend form.

B6. Centralized Error Handling & 404s

Add a catch-all 404 handler for undefined routes and a global Express error-handling middleware so no error ever crashes the server or returns raw HTML/stack traces.

Evaluation criteria:

- Requesting an undefined route (e.g., GET /api/doesnotexist) returns HTTP 404 with a JSON error body.
- Deliberately triggering a server error (e.g., malformed request) is caught and returns a JSON error body with an appropriate status code, and the server keeps running afterward.

B7. CORS & Environment Configuration

Enable CORS so the React development server can call this API, and load all configuration (port, data file path, allowed origin) from environment variables via dotenv.

Evaluation criteria:

- Frontend fetch/axios calls from the React dev server succeed with no CORS errors in the browser console.
- A .env.example file is included listing every required variable, with no secrets committed in .env itself.

4. Tasks — Frontend Integration (React)

These tasks modify your existing Assignment 2 components to consume the backend built above.

F1. Fetch Projects from the Backend

Replace the static import of src/data/projects.js on the Projects page with a call to GET /api/projects made inside a useEffect hook, storing the result in state.

Evaluation criteria:

- The Projects page renders cards using only data returned from the API — the static data file is no longer imported for rendering.
- A loading state is shown while the request is pending, using a new piece of state introduced specifically for this purpose.

F2. Handle Fetch Errors Gracefully

Add error-state handling so that if the backend is unreachable or returns an error, the user sees a clear message instead of a blank or broken page.

Evaluation criteria:

- Stopping the backend server and reloading the Projects page shows a visible error message in the UI (not a blank page or unhandled console error only).
- Restarting the backend and reloading recovers normal behaviour without a code change.

F3. Fetch a Single Project for the Detail Page

Update the `/projects/:projectId` detail page to fetch that specific project from `GET /api/projects/:id` (using `useParams` to read the id) instead of filtering a local in-memory array.

Evaluation criteria:

- Directly visiting a project's detail URL (a deep link, e.g., after a manual refresh) correctly loads and displays that project via the API.
- Visiting a detail URL with a non-existent id shows a clear "project not found" message rather than a crash or blank page.

F4. Submit the Contact Form to the Backend

Update the Contact form's submit handler to send its data via POST to `/api/contact` instead of only performing client-side validation. Display the outcome to the user.

Evaluation criteria:

- A valid submission shows a success confirmation and resets the form fields.
- If the server rejects the submission (e.g., by bypassing client validation via browser dev tools), the server-provided error message is displayed to the user.

5. Deliverables

- Updated GitHub repository containing both the existing React frontend and the new `/server` backend (or a linked backend repo, clearly referenced in the README).
- Updated README.md with: backend setup/run instructions, the full list of API endpoints with sample requests/responses, and a `.env.example`.
- A Postman collection (exported JSON) or a set of curl commands covering all seven backend endpoints (B1–B7), including at least one failure case per validated endpoint.

A short screen recording (2-3 minutes) showing: the Projects page loading data from the live backend, a deep link to a project detail page, the contact form being submitted successfully, and the frontend's error state after the backend is stopped.

6. Constraints / Additional Information

- Backend must live in a `/server` folder inside the same repository as your Assignment 2 frontend (or be clearly linked from the README if kept separate).
- Use plain fetch or axios for HTTP calls — data-fetching libraries such as React Query/SWR are not permitted for this assignment, since the objective is to demonstrate `useEffect`-based data fetching directly.
- No ORM or database is mandatory; an in-memory array or JSON file is sufficient for storage, but the choice must be stated in the README.
- All previously working Assignment 2 functionality (theme toggle, routing, 404 page, responsive layout) must continue to work unmodified.

- The application must run with two documented commands — one to start the backend, one to start the frontend — with zero unhandled console errors.

7. Academic Integrity

This is an individual assignment. Submissions will be checked for plagiarism (including AI-generated boilerplate) using [MOSS/Turnitin]. Use of AI coding assistants for full solution generation is prohibited; assistance for debugging small snippets must be disclosed in the README. Violations will be referred to the Institute's Academic Ethics Committee as per institute policy.

8. Late Submission Policy

- Up to 24 hours late: 10% penalty
- Up to 48 hours late: 25% penalty
- Beyond 48 hours: not accepted without prior instructor approval